Vocabulaire :

- Les arbres et le vocabulaire associé / arbre binaire de recherche (ABR) / efficacité / spécification d'un programme / postcondition / parcours préfixe / parcours infixe / parcours suffixe / parcours en largeur / programmation itérative.

Prérequis :

- Manipulation des structures de liste et des fonctions. Les listes simplement et doublement chaînée en POO.
- La récursivité.
- Les parcours.
- Avoir fait les séances précédentes sur les arbres et maîtrisé le vocabulaire.

Objectifs :

- Savoir l'efficacité é d'un algorithme de recherche dans un ABR suivant sa forme.
- Savoir qu'une complexité en Log(N) est considéré comme efficace.
- Implémenter des méthodes en étant guider et comprendre tous les algorithmes pour parcourir un arbre : en profondeur (préfixe, infixe et suffixe) et en largeur.
- Implémenter un algorithme de recherche d'un élément dans un ABR.

Pour cette séance,
surtout reste calme !

Cette séance est la cinquième concernant la structure de données **arbres**.

Il y aura des arbres pour l'examen du bac donc aujourd'hui, c'est entraînement.

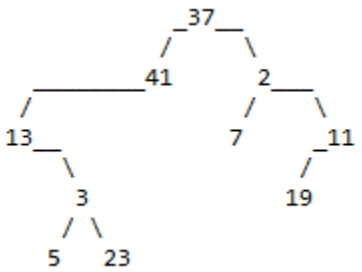
Voici les règles du jeu : tu fais le maximum, jusqu'au bout de la partie I, sans regarder les séances précédentes. Seulement à la fin de l'énoncé, tu t'appuies sur les cours précédents pour compléter tes lacunes. Bon courage !

I- ENTRAÎNEMENT SUR LES ARBRES

Partie écrite

On donne ci-dessous la représentation informatique de l'arbre a et sa representation graphique. La classe Arbre crée simplement un nœud racine dont la clé est définie en paramètre et dont les sous-arbres gauche (Sag) et droit (Sad) sont vides.

```
a = Arbre(37) # Créé une racine de clé 37
a.Sag = Arbre(41) # Crée un fils gauche à la racine
a.Sag.Sag = Arbre(13)
a.Sag.Sag.Sad= Arbre(3)
a.Sag.Sag.Sad.Sag= Arbre(5)
a.Sag.Sag.Sad.Sad = Arbre(23)
a.Sad = Arbre(2)
a.Sad.Sag = Arbre(7)
```



1) Il manque des lignes de code pour que l'arbre soit completdans. Lesquelles ?

[Copie-colle ton code ici et sauvegarde ensuite ton fichier pdf.]

2) À quelle sous-famille appartient cet arbre ? Justifie.

[Écris la réponse ici]

3) Que signifie ABR ?

[Écris la réponse ici]

4) Quel nom donne-t-on aux extrémités basses d'un arbre ?

[Écris la réponse ici]

5) Quelle est la taille de cet arbre ? Justifie.

[Écris la réponse ici]

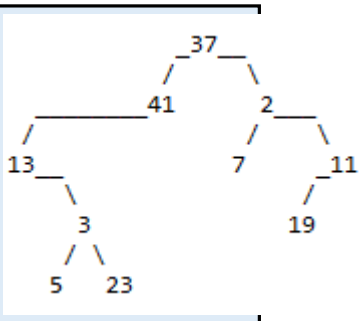
6) Quelle est la hauteur de cet arbre ? Justifie.

[Écris la réponse ici]

7) Donne le résultat du parcours en largeur de cet arbre.

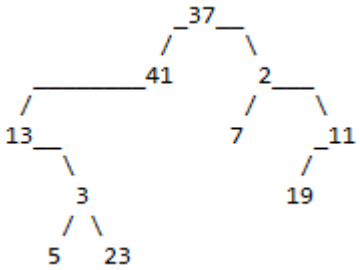
[Écris la réponse ici]

8) Donne les résultats des parcours préfixe, infixe et suffixe.



9) Donne la représentation de l'arbre en tableau 2D sans indiquer les 0.

	2	3	5	7	11	13	19	23	37	41
2										
3										
5										
7										
11										
13										
19										
23										
37										
41										



10) Donne la représentation de l'arbre en liste d'adjacence.

2	3	5	7	11	13	19	23	37	41

Partie pratique

Un programme intitulé `C33_ExerciceBilan_eleve.py` correspond à une ébauche de l'implémentaton Python des algorithmes relatifs aux arbres vus en classe.

Complète les différentes méthodes de la class Arbre.

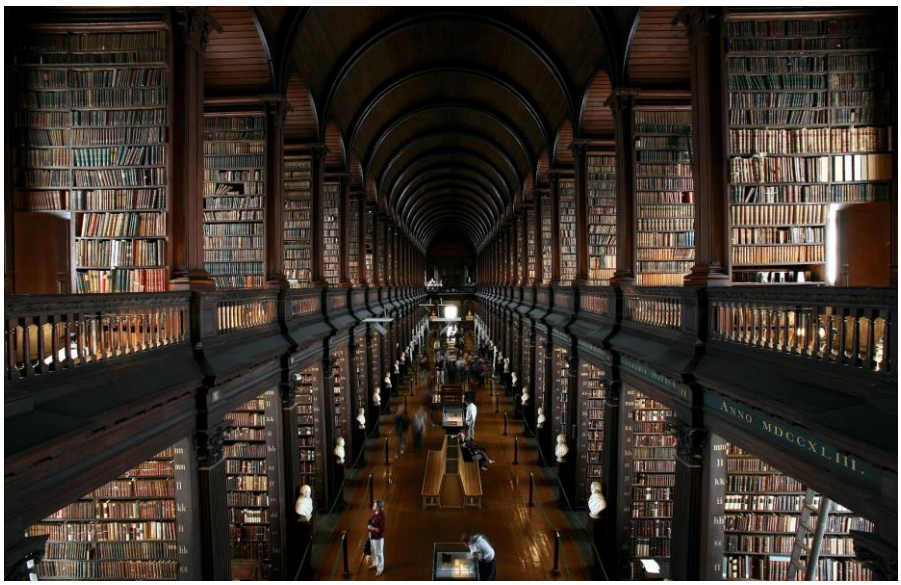
Un jeu de test peut être décommenté en bas du programme.

FIN DE L'ENTAINEMENT

À faire pour la prochaine séance

III- RECHERCHE D'UN ÉLÉMENT DANS UN ABR, LE RETOUR

Imaginons une bibliothèque qui contienne vraiment beaucoup de livres, répartis dans 17 576 salles reliées les unes aux autres par des portes. Chaque salle contient une porte d'entrée et, possiblement, deux portes de sortie vers deux autres salles. Pour retrouver facilement l'emplacement d'un livre, les bibliothécaires astucieux ont eu l'idée suivante. Dont la toute première salle, dans laquelle on entre par la porte d'entrée la bibliothèque, ils ont mis tous les ouvrages dont le titre commence par MDB. Si en revanche le titre commence par trois lettres venant avant MDB dans l'ordre alphabétique, alors il faut prendre la porte de sortie de gauche. Sinon, il faut prendre la porte de sortie de droite. On continue alors la recherche dans la salle suivante. Si on est allé dans la salle de gauche, par exemple, on y trouve tous les ouvrages dont le titre commence par GBN. Là, de même, si le titre commence par trois lettres venant avant GBN dans l'ordre alphabétique, alors il faut prendre la porte de sortie gauche, sinon la porte de droite. Et ainsi de suite jusqu'à ce qu'on parvienne à la salle contenant



tous les livres dont les trois premières lettres du titre sont celles du titre qui nous intéresse. Une telle organisation est incroyablement efficace. En effet, avec la répartition choisie par les bibliothécaires, il ne faut jamais traverser plus de 15 salles pour trouver un ouvrage, alors qu'il y en a, rappelons-le, 17 576 salles au total.
Tu l'as compris, on peut visualiser les salles de cette bibliothèque comme un arbre binaire de recherche (ABR) où chaque nœud correspond à une salle.

Ce qui nous intéresse ici, c'est la complexité temporelle (= efficacité) d'un algorithme de recherche d'un élément dans un arbre ABR. Elle dépend d'un paramètre important, la forme de l'arbre.

Cas 1 : l'arbre ABR est relativement équilibré

Si les N éléments de l'arbre sont répartis à peu près équitablement entre les deux sous-arbres, l'algorithme élimine la moitié des éléments à chaque étape.

Cela fait écho à la recherche dichotomique dans un tableau trié que nous avons étudié en 1ère NSI en fin d'année. Pour rappel, voici le pseudo-code correspondant :

Recherche dichotomique de l'élément e dans un tableau trié par ordre croissant tab de longueur n.	
Pseudo-code	**Commentaires**
`deb = 0`	Au départ l'intervalle de recherche correspond à la totalité du tableau.
`fin = n - 1`	La borne de gauche `deb` est initialisée à l'indice minimum du tableau.
`tant que deb < fin faire`	La borne de droite `fin` est initialisée à l'indice maximum du tableau.
`  m = (deb + fin) // 2`	La borne de gauche doit être strictement plus petite que la borne de droite. Partie entière du milieu de l'intervalle.
`  si tab[m] = e alors`	On a trouvé l'élément `e` dans le tableau à la position `m`
`  retourner m`	
`  si tab[m] > e alors`	Dans ce cas, on continue à chercher dans la première moitié de l'intervalle précédent soit `[deb ; m - 1]`.
`  fin = m - 1`	
`  sinon deb = m+1`	Dans ce cas, on continue à chercher dans la moitié droite de l'intervalle précédent soit `[m + 1 ; fin]`.
`retourner « Non trouvé »`	On est sorti de la boucle sans trouver l'élément e.

Vous devez retenir que dans le cas d'un ABR relativement équilibré, la complexité temporelle est en O(N)=log N.

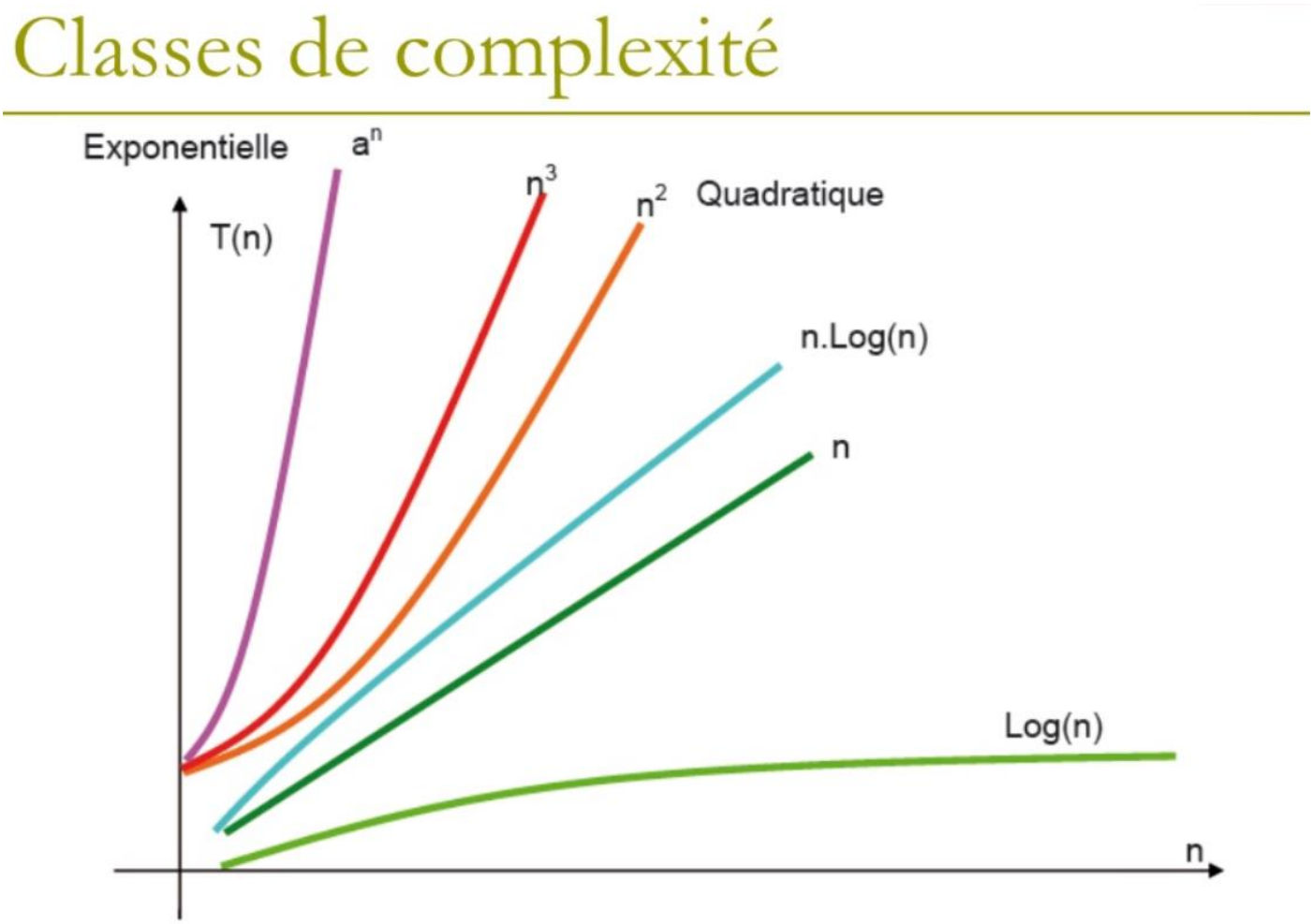
Cas 2 : l'arbre ABR est un peigne ou tout en longueur

Si la bibliothèque est une enfilade de pièce, dans ce cas, on se retrouve à devoir dans le pire des cas traverser les N salles pour obtenir l'ouvrage se trouvant dans la dernière salle.

Vous devez retenir que dans le cas d'un ABR sous forme de peigne ou tout en longueur (=chaque nœud a un seul fils ou presque), la complexité temporelle est en O(N)= N.

Comparaison des complexités

Voici un graphique qui permet de bien comprendre que **la complexité en Log(N) est considérée comme très efficace**, notamment en comparaison par rapport à une complexité de N.
Rappelons que l'on se place toujours sur des grandes valeurs de N pour juger de l'efficacité d'un algorithme.



Pour finir, voici le code permettant de rechercher un élément dans un ABR.

Vois comme il est similaire au pseudo-code de la recherche dichotomique dans un tableau trié.

Version récursive

```
def rechercherElement(self, e):
    if self.estVide(): return False
    if e == self.info: return True
    if e < self.info:
        if self.fg:
            return self.fg.rechercherElement(e)
        else:
            return False
    if e > self.info:
        if self.fd:
            return self.fd.rechercherElement(e)
        else:
            return False
```

Version itérative

```
def rechercherElementIteratif(self, e):
    if self.estVide(): return False
    f = pf.File([])
    f.enfiler(self)
    while not f.estVide():
        n = f.traiter()
        if n.info == e: return True
        if n.fg: f.enfiler(n.fg)
        if n.fd: f.enfiler(n.fd)
    return False
```